A Separation Logic Formalization of the C++ Mutex Library

Ke Du

10/28/2026

std::mutex for mutual exclusion

mutex ensures mutual exclusion

```
unsigned int x = 0;
std::mutex m{};

void add_two() {
  m.lock();
  x += 2;
  m.unlock();
}
```

Running add_two concurrently

```
int main() {
  std::thread t(add_two);
  add_two();
  t.join();
}
```

While one caller holds m, the other waits at `m.lock()`.

Lock specifications in separation logic

$$\{\ell \mapsto \mathsf{lock}(P)\} \quad \texttt{lock}(\ell) \quad \{\ell \mapsto \mathsf{lock}(P) * \mathsf{locked}(\ell) * P\}$$

$$\{\ell \mapsto \mathsf{lock}(P) * \mathsf{locked}(\ell) * P\} \quad \texttt{unlock}(\ell) \quad \{\ell \mapsto \mathsf{lock}(P)\}$$

$\ell \mapsto \mathsf{locked}(\ell)$ says l stores a lock, protecting resource P.

Cross-thread unlock

Thread L

```
acquire(a);
*x = *x + 1;
release(b);
```

Thread R

```
acquire(b);
*y = *x + 1;
release(a);
```

Pass $\mathsf{locked}(\mathtt{a})$ and x to thread R with lock b.

Works in heap lang, forbidden in C++.

recursive_mutex: can acquire multiple times

```
unsigned long balance_a, balance_b;
std::recursive_mutex m{};
```

Update the first balance

```
void update_a(long x) {
  m.lock();
  balance_a += x;
  m.unlock();
}
```

Update the second balance

```
void update_b(long x) {
  m.lock();
  balance_b += x;
  m.unlock();
}
```

Atomic Account Transfer With Recursive Mutex

Outer call · lock count starts at 0

```
void transfer(int x) {
  m.lock();              // 0 -> 1
  update_a(x);           // 1 -> 2 -> 1
  update_b(-(long)x);    // 1 -> 2 -> 1
  m.unlock();            // 1 -> 0
}
```

The thread owns m while its lock count is positive.

shared_mutex: Reader-Writer Lock

```
bool resource = false;
std::shared_mutex m{};

void set_resource() {
  m.lock();
  resource = true;
  m.unlock();
}
```

While this writer holds m, readers and other writers wait.

shared_mutex: Reader-Writer Lock

```
void await_resource() {
  for (;;) {
    m.lock_shared();
    bool ret = resource;
    m.unlock_shared();
    if (ret)
      break;
  }
}
```

Several `await_resource` calls may read at once.

lock_guard: RAII wrapper of a BasicLockable object

```
std::mutex m;
int res;

void f() {
  std::lock_guard<std::mutex> lock(m);
 // critical section
}
```

Named requirements: informal C++ contracts.

`m.lock()` : Blocks until a lock can be acquired for the current thread.

`m.unlock()` : Unlocks the lock.

Requirements revisited

Unlock	by a thread that did not acquire the mutex
Reacquire	an ordinary mutex already owned by this thread
Destroy	a mutex while any thread holds it

> Violation: **undefined behavior in C++.**

Contribution

Thread-aware specs for mutexes, RAII wrappers

Formalize "BasicLockable"

Reference implementations for mutexes, satisfy our specs and passed libC++ test suite

Case study: data structure predicate that uses lock under the hood is not tied to a thread and still transferrable.

BRiCk: an Iris-based program logic for C++

C++ support class, template. Translates C++ to an AST defined in Rocq.

Iris based Ghost state, invariants, WP. No semantics, axiomatized proof rules.

Thread-local resource Every predicate is implicitly indexed by a thread ID.

$\mathsf{tid}(t)$ a special proposition that says **"the current thread is t."**. Cannot transfer between threads.

Other (primitive) assertions can move between threads, like in most CSLs.

std::mutex: lock and unlock

N: **namespace**; q, π: fraction.

$$m \mapsto_{\pi_1} \mathsf{mutex}_{N,\gamma}(P) * m \mapsto_{\pi_2} \mathsf{mutex}_{N,\gamma}(P) \dashv\vdash m \mapsto_{\pi_1+\pi_2} \mathsf{mutex}_{N,\gamma}(P) \qquad (\pi_1 + \pi_2 \leq 1)$$

$$\{\mathsf{tid}(t) * m \mapsto_{\pi} \mathsf{mutex}_{N,\gamma}(P) * \mathsf{not_locked}_{N,\gamma}(t, q)\}$$

`m.lock()`

$$\{\mathsf{tid}(t) * m \mapsto_{\pi} \mathsf{mutex}_{N,\gamma}(P) * \mathsf{m_locked}_{N,\gamma}(t, q) * P\}$$

$$\{\mathsf{tid}(t) * m \mapsto_{\pi} \mathsf{mutex}_{N,\gamma}(P) * \mathsf{m_locked}_{N,\gamma}(t, q) * \triangleright P\}$$

`m.unlock()`

$$\{\mathsf{tid}(t) * m \mapsto_{\pi} \mathsf{mutex}_{N,\gamma}(P) * \mathsf{not_locked}_{N,\gamma}(t, q)\}$$

std::mutex: constructor and destructor

$$\{\triangleright P\}$$

$$\texttt{mutex()}$$

$$\{m.\ \exists \gamma.\ m \mapsto \mathsf{mutex}_{N,\gamma}(P) * \mathsf{token}_{\gamma}(1)\}$$

$$\{m \mapsto \mathsf{mutex}_{N,\gamma}(P) * \mathsf{token}_{\gamma}(1)\}$$

$$\texttt{m.}\sim\texttt{mutex()}$$

$$\{P\}$$

Registration rules

$\mathsf{mutex_perm}_t(A)$ says thread t can register for mutex that has namespace A. A namespace is a set. A thread is given $\mathsf{mutex_perm}_t(\top)$ when spawn.

$$\mathsf{mutex_perm}_t(A \uplus B) \dashv\vdash \mathsf{mutex_perm}_t(A) * \mathsf{mutex_perm}_t(B)$$

where $A \cap B = \varnothing$.

$$\mathsf{mutex_perm}_t(N) * \mathsf{token}_\gamma(q) \underset{\text{unregister}}{\overset{\text{register}}{\rightleftarrows}} \mathsf{not_locked}_{N,\gamma}(t, q)$$

Specs should ensure that

✓ **Unlock**	Only the thread that acquired the mutex may unlock it.
✓ **Reacquire**	Forbidden if this thread already owns the same ordinary mutex.
✓ **Destroy**	Only when no thread holds the mutex.

$$\mathsf{m_locked}_{N,\gamma}(t, q)$$

Verifying add_two

```
unsigned int x = 0;
std::mutex m{};
```
// $P \triangleq \exists n.\, x \mapsto n * \mathsf{even}(n)$

$\{\mathsf{tid}(t) * m \mapsto_{1/2} \mathsf{mutex}_{N,\gamma}(P) * \mathsf{token}_\gamma(1/2) * \mathsf{mutex_perm}_t(N)\}$

```
void add_two() {
```
$\Rightarrow \{\mathsf{tid}(t) * m \mapsto_{1/2} \mathsf{mutex}_{N,\gamma}(P) * \mathsf{not_locked}_{N,\gamma}(t, 1/2)\}$

```
  m.lock();
```
$\{\ldots * P * \mathsf{m_locked}_{N,\gamma}(t, 1/2)\}$

```
  x += 2;
```
$\{\ldots * P * \mathsf{m_locked}_{N,\gamma}(t, 1/2)\}$

```
  m.unlock();
```
$\{\mathsf{tid}(t) * m \mapsto_{1/2} \mathsf{mutex}_{N,\gamma}(P) * \mathsf{not_locked}_{N,\gamma}(t, 1/2)\} \Rightarrow$

```
}
```
$\{\mathsf{tid}(t) * m \mapsto_{1/2} \mathsf{mutex}_{N,\gamma}(P) * \mathsf{token}_\gamma(1/2) * \mathsf{mutex_perm}_t(N)\}$

Verifying the two-thread client

```
int main() {
```
$\{\mathsf{tid}(t_0) * m \mapsto \mathsf{mutex}_{N,\gamma}(P) * \mathsf{token}_\gamma(1) * \mathsf{mutex_perm}_{t_0}(\top)\}$
```
  std::thread t(add_two);
```
$\{\mathsf{tid}(t_0) * m \mapsto_{1/2} \mathsf{mutex}_{N,\gamma}(P) * \mathsf{token}_\gamma(1/2) * \mathsf{mutex_perm}_{t_0}(\top)\}$

$\Rightarrow \{\ldots * \mathsf{mutex_perm}_{t_0}(N) * \mathsf{mutex_perm}_{t_0}(\top \setminus N)\}$
```
  add_two();
```
$\{\ldots * \mathsf{mutex_perm}_{t_0}(N) * \mathsf{mutex_perm}_{t_0}(\top \setminus N)\}$

$\Rightarrow \{\mathsf{tid}(t_0) * m \mapsto_{1/2} \mathsf{mutex}_{N,\gamma}(P) * \mathsf{token}_\gamma(1/2) * \mathsf{mutex_perm}_{t_0}(\top)\}$
```
  t.join();
```
$\{\mathsf{tid}(t_0) * m \mapsto \mathsf{mutex}_{N,\gamma}(P) * \mathsf{token}_\gamma(1) * \mathsf{mutex_perm}_{t_0}(\top)\}$
```
}
```

std::recursive_mutex

```
std::recursive_mutex rm;

// not locked
rm.lock();   // acquired
rm.lock();   // same thread!
...
rm.unlock();
rm.unlock(); // really released the lock
```

The $holds$ predicate

An ordinary mutex uses $\mathsf{m_locked}_{N,\gamma}(t, q) * P$.

A recursive mutex uses $\mathsf{holds}_{N,\gamma}(t, n, q, P)$; n is the acquisition depth.

$$\mathsf{holds}_{N,\gamma}(t, 0, q, P) \triangleq \mathsf{not_locked}_{N,\gamma}(t, q)$$

$$\frac{n > 0}{\mathsf{holds}_{N,\gamma}(t, n, q, P) \vdash P * (P \mathrel{-\!\!*} \mathsf{holds}_{N,\gamma}(t, n, q, P))}$$

Return P to recover `holds`.
An existential witness inside P is hidden again.

Recursive mutex rules

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{rmutex}_{N,\gamma}(P) * \mathsf{holds}_{N,\gamma}(t, n, q, P)\}$$

$$\texttt{m.lock()}$$

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{rmutex}_{N,\gamma}(P) * \mathsf{holds}_{N,\gamma}(t, n+1, q, P)\}$$

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{rmutex}_{N,\gamma}(P) * \mathsf{holds}_{N,\gamma}(t, n+1, q, P)\}$$

$$\texttt{m.unlock()}$$

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{rmutex}_{N,\gamma}(P) * \mathsf{holds}_{N,\gamma}(t, n, q, P)\}$$

Problem: P cannot change

$\{\ulcorner n > 0 \urcorner * \ldots * \mathsf{holds}_{N,\gamma}(t, n, q, \exists v.\ x \mapsto v)\}$

```
m.lock();
```

$\{\ldots * \mathsf{holds}_{N,\gamma}(t, n+1, q, \exists v.\ x \mapsto v)\}$

$\Rightarrow \{\ldots * x \mapsto a\}$

```
x++;
```

$\{\ldots * x \mapsto a + 1\}$

$\Rightarrow \{\ldots * \mathsf{holds}_{N,\gamma}(t, n+1, q, \exists v.\ x \mapsto v)\}$

```
m.unlock();
```

$\{\ldots * \mathsf{holds}_{N,\gamma}(t, n, q, \exists v.\ x \mapsto v)\}$

The holds predicate

$$\mathsf{holds}_{N,\gamma}(t, n, q, P, \boxed{a})$$

instead of $P : PRED$, we split it into the fixed $P : A \to PRED$ and the changing $a : A$.

$$\mathsf{holds}_{N,\gamma}(t, 0, q, P, \boxed{a}) \triangleq \mathsf{not_locked}_{N,\gamma}(t, q)$$

$$\frac{n > 0}{\mathsf{holds}_{N,\gamma}(t, n, q, P, \boxed{a}) \vdash \boxed{P(a)} * \left(\boxed{\forall b.\ P(b)} \mathrel{-\!\!*} \mathsf{holds}_{N,\gamma}(t, n, q, P, \boxed{b})\right)}$$

At depth 0, a is ignored.

Recursive mutex: lock and unlock

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{rmutex}_{N,\gamma}(P) * \mathsf{holds}_{N,\gamma}(t, n, q, P, \boxed{a})\}$$

`m.lock()`

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{rmutex}_{N,\gamma}(P) * \boxed{\exists a'.}\ \mathsf{holds}_{N,\gamma}(t, n+1, q, P, \boxed{a'}) * \boxed{\ulcorner n > 0 \Rightarrow a' = a \urcorner}\}$$

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{rmutex}_{N,\gamma}(P) * \mathsf{holds}_{N,\gamma}(t, n+1, q, P, \boxed{a})\}$$

`m.unlock()`

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{rmutex}_{N,\gamma}(P) * \mathsf{holds}_{N,\gamma}(t, n, q, P, \boxed{a})\}$$

Remember a while holding the recursive mutex

$\{\ulcorner n > 0 \urcorner * \ldots * \mathsf{holds}_{N,\gamma}(t, n, q, \lambda v.\ x \mapsto v, a)\}$

```
m.lock();
```

$\{\ldots * \mathsf{holds}_{N,\gamma}(t, n+1, q, \lambda v.\ x \mapsto v, a)\}$

$\Rightarrow \{\ldots * x \mapsto a\}$

```
x++;
```

$\{\ldots * x \mapsto a + 1\}$

$\Rightarrow \{\ldots * \mathsf{holds}_{N,\gamma}(t, n+1, q, \lambda v.\ x \mapsto v, a + 1)\}$

```
m.unlock();
```

$\{\ldots * \mathsf{holds}_{N,\gamma}(t, n, q, \lambda v.\ x \mapsto v, a + 1)\}$

shared_mutex: take a fractional `P`

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{smutex}_{N,\gamma}(P) * \mathsf{not_locked}_{N,\gamma}(t, q)\}$$

`m.lock()`

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{smutex}_{N,\gamma}(P) * P(1) * \mathsf{sm_locked}_{N,\gamma}(t, q)\}$$

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{smutex}_{N,\gamma}(P) * \mathsf{sm_locked}_{N,\gamma}(t, q) * \triangleright P(1)\}$$

`m.unlock()`

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{smutex}_{N,\gamma}(P) * \mathsf{not_locked}_{N,\gamma}(t, q)\}$$

P is a fractional predicate; exclusive acquisition yields $P(1)$.

shared_mutex: take a fractional `P`

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{smutex}_{N,\gamma}(P) * \mathsf{not_locked}_{N,\gamma}(t, q)\}$$

`m.lock_shared()`

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{smutex}_{N,\gamma}(P) * \exists q_P.\ \mathsf{sm_reader_locked}_{N,\gamma}(t, q, q_P) * P(q_P)\}$$

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{smutex}_{N,\gamma}(P) * \mathsf{sm_reader_locked}_{N,\gamma}(t, q, q_P) * \triangleright P(q_P)\}$$

`m.unlock_shared()`

$$\{\mathsf{tid}(t) * m \mapsto_\pi \mathsf{smutex}_{N,\gamma}(P) * \mathsf{not_locked}_{N,\gamma}(t, q)\}$$

Token fraction q and protected-resource fraction q_P are separate.

mutex_perm: permission for thread t to "use" the namespace N

```
int main() {
  {tid(t_0) * m ↦ data_structure_handle_N * mutex_perm_t0(⊤)}
  std::thread t(op);
  {tid(t_0) * m ↦_1/2 data_structure_handle_N * mutex_perm_t0(⊤)}
  ⇒ {... * mutex_perm_t0(N) * mutex_perm_t0(⊤ \ N)}
  op();
...
}
```

Assume `op` a concurrent operation, implemented by C++ concurrency primitives. Can still distribute its handle without thinking about thread ID; operation's precondition only require the corresponding $mutex_perm$.

BasicLockable: modeling generic lock object

```
std::mutex m;
int size;

bool get_size() {
  std::lock_guard<std::mutex> lock(m);
  return size;
}
```

`lock_guard` is generic over a *BasicLockable* object.

BasicLockable: modeling generic lock object

C++ requires BasicLockable to have these operations:

- `lock()` : blocks until the calling thread acquires the "lock".
- `unlock()` : the calling thread must own the lock; releases the lock.

BasicLockable: modeling generic lock object

Formally, an object is BasicLockable if we can define `{ R; do_lock; do_unlock }`

With that, we automatically have:

$$\{p \mapsto R(q, m) * \mathsf{do_lock}(p, m, K)\}$$

`p.lock()`

$$\{p \mapsto R(q, m) * K\}$$

$$\{p \mapsto R(q, m) * \mathsf{do_unlock}(p, m, K)\}$$

`p.unlock()`

$$\{p \mapsto R(q, m) * K\}$$

std::lock_guard specifications

$$\{p \mapsto R(q, m) * \mathsf{do_lock}(p, m, K)\}$$

`lock_guard(p)`

$$\{g.\ g \mapsto \mathsf{lguard}(p, q, m) * K\}$$

$$\{g \mapsto \mathsf{lguard}(p, q, m) * \mathsf{do_unlock}(p, m, K)\}$$

`g.~lock_guard()`

$$\{K * p \mapsto R(q, m)\}$$

Related work

Work	
VerCors Haack et al. (2008); Amighi et al. (2013)	Tracks threads, but is hard to transfer resource (or even impossible), whereas in BRiCk `tid` is the only non-transferrable one. Two specs for locking a recursive mutex, depending on whether a thread holds it or not.
Iris based CSLs VST, RustBelt, RefinedC …	Does not track thread ID.
VeriFast Verifies Rust stdlib, including Rust mutex proof	Tracks thread ID, but lock predicate properties are axiomatized instead of proved from ghost states.

Conclusion

Specs for mutexes and RAII wrappers. General locking/unlocking via BasicLockable. Verified reference implementations for the locks.

A modular mutex perm ghost state, an abstraction of the permission to acquire a "mutex-like" data structure. Decouples data structure distribution and mutex permission.

Specs and ghost states portable to other logic with a notion of `tid`.

Specs designed manually, proof assisted by AI.

Future work

Other concurrency primitives	specifications for condition variables, futures, and RCU.
Implementation verification	Formalize the interfaces of OS primitives such as futex, verify GCC/Clang mutex implementations.
User program verification	Verify real-world programs that use C++ concurrency libraries with our specs.

On going project, join us!